

From paper to pitch, in minutes.

SYSTEM OVERVIEW

Three-tier architecture

Next.js frontend	FastAPI gateway	Pipeline workers	
Vercel · Clerk auth SSE client · file upload	Cloud Run · validates routes + rate limits	Cloud Run Jobs async agent execution	
Supabase PostgreSQL + Realtime	Redis Job queue + cache	GCS PDF + output storage	ChromaDB Vector embeddings

CHECKPOINT + RESUME

Zero-loss pipeline state

Every agent writes its output to Supabase before the next one starts. Pipeline dies at agent 3? Restart picks up at agent 3. You never re-run completed work. You never re-charge API costs for work already done.

pipeline_runs table

id (uuid) · project_id (uuid FK) · agent_name (text) status ENUM: pending | running | completed | failed attempt_count (int) · input_snapshot (jsonb) output_snapshot (jsonb) · error_message (text) model_used (text) · started_at · completed_at checksum (text) ← hash of input, detects stale cache

On every pipeline run, before calling an agent, the system checks: does a **completed** row exist for this agent + project + matching checksum? Yes → skip, use cached output. No → run it, write result, mark completed.

RETRY LOGIC

Three levels of failure handling

Level 1	Transient failure — auto retry	API rate limit, network timeout, 503. Exponential backoff: 2s → 4s → 8s. Max 3 attempts.
Level 2	Agent failure — fallback model	Claude API fails after 3 retries → switch to Gemini automatically. Same prompt, different model. User never knows.
Level 3	Hard failure — dead letter queue	Both models fail. Job moves to Redis dead letter queue. Pipeline status: failed. Last checkpoint preserved. User sees retry button. You get email alert.

JOB QUEUE ARCHITECTURE

Redis + ARQ — non-blocking pipeline

FastAPI gateway never runs agents directly. It enqueues a job and returns 202 immediately. Workers pick up jobs asynchronously. 100 concurrent users = 100 jobs in queue, not 100 blocking threads.

POST /api/pipeline/run/{project_id} → validates request → creates pipeline_run rows (one per agent, status: pending) → enqueues job in Redis: {project_id, file_path} → returns 202 Accepted immediately ARQ worker picks up job: → checks checkpoint table → runs only pending/failed agents → writes each output to Supabase → publishes status event to Supabase Realtime → frontend receives live update via websocket

MODEL ROUTING

Claude + Gemini — no single point of failure

Claude Sonnet 4.6 Primary model	Gemini 2.0 Flash Secondary / fallback
Paper reader · Feasibility checker · Pitch builder. Complex reasoning tasks where output quality is critical.	Market scout · Domain classification · Fallback for all agents. Faster, cheaper, sufficient for search synthesis.

FOLDER STRUCTURE

Complete backend layout

```
lemma-backend/ ███ main.py ███ worker.py ← ARQ worker entry point ███
requirements.txt ███ Dockerfile ███ .env.example ███ api/ █ ███ routes/ █ █ ███
projects.py █ █ ███ pipeline.py █ █ ███ exports.py █ ███ middleware/ █ █ ███
auth.py ← Clerk JWT validation █ █ ███ rate_limit.py █ ███ models/schemas.py ███
agents/ █ ███ base.py ← BaseAgent class █ ███ paper_reader.py █ ███
market_scout.py █ ███ feasibility.py █ ███ investor_finder.py █ ███
pitch_builder.py ███ core/ █ ███ pdf_parser.py █ ███ pipeline.py ← orchestrator █
████ checkpoint.py ← read/write checkpoints █ ███ model_router.py ← Claude/Gemini
routing █ ███ retry.py ← exponential backoff ███ db/ █ ███ supabase.py █ ███
migrations/001_initial.sql ███ storage/gcs.py ███ jobs/pipeline_job.py ← ARQ job
definition ███ tests/
```

DATABASE SCHEMA

Four Supabase tables

projects	pipeline_runs	agent_outputs	exports
id uuid PK user_id (Clerk) title text institution text lab text domain text file_path text status text created_at	id uuid PK project_id FK agent_name status attempt_count input_snapshot jsonb output_snapshot jsonb error_message model_used checksum	id uuid PK project_id FK agent_name output_json jsonb version int created_at	id uuid PK project_id FK format (pdf/pptx/docx) file_path created_at

API ENDPOINTS

Complete route map

POST	/api/projects/create	Create project, store metadata in Supabase
POST	/api/pipeline/run/{id}	Accept PDF upload, enqueue job, return 202
GET	/api/pipeline/stream/{id}	SSE stream — live agent status updates
GET	/api/projects/{id}/results	Full results JSON from all agents
GET	/api/projects/	List all projects for authenticated user
POST	/api/pipeline/retry/{id}	Retry failed agents from last checkpoint
GET	/api/exports/{id}/{format}	Download PDF, PPTX, or DOCX output

